

原码一位乘

① 分析笔算乘法

$A = -0.1101$ $B = 0.1011$

要求

解决

$$A \times B = 0.10001111$$

$$\begin{array}{r} 0.1101 \\ \times 0.1011 \\ \hline 1101 \\ 1101 \\ 0000 \\ 1101 \\ \hline 0.10001111 \end{array}$$

乘积符号由心算求得

符号位单独处理✓

乘数的某一位(1或0)决定是否加被乘数✓

4个位积一起错位相加?

乘积的位数扩大1倍✓

$A_f \oplus B_f$

与运算

(将乘数每个数位与被乘数绝对值相与,可得4个不同的位积)

A, B分别用一个寄存器存放, 用两个寄存器

② 笔算乘法改进

$$\begin{aligned} A \cdot B &= A \cdot 0.1011 \\ &= 0.1A + 0.00A + 0.001A + 0.0001A \\ &= 0.1(A + 0.00A + 0.001(A + 0.1A)) \\ &= 0.1(A + 0.01[0.1A + 0.1(A + 0.1A)]) \\ &= 0.1\{A + 0.1[0.1A + 0.1(A + 0.1A)]\} \end{aligned}$$

错位相加, 可转化为

加, 右移, 加, 右移... = $2^{-1}\{A \oplus 2^{-1}[0.1A \oplus 2^{-1}(A \oplus 2^{-1}(A \oplus 0))]\}$ "x2⁻¹" 相当于右移1位

交替进行

步骤

- ① 被乘数 A + 0
- ② 右移1位, 得新的部分积
- ③ 部分积 + 被乘数 A · (0/1)
- ...
- ⑧ 右移1位, 得到结果

乘积的有效数位, 下次加时不参运算, 但不可丢弃

乘法过程(竖式)	寄存器1 部分积	寄存器2 乘数	说明
	0.0000	1011	乘过了, 没用, 可移走
+	0.1101		初态, 部分积=0
	0.1101		乘数是1, 加被乘数
+	0.0110		
	0.1101		
	1.0011		右移1位, 形成新部分积
+	0.1001		乘数是1, 加被乘数
	0.0000		
+	0.1001		右移1位, 形成新部分积
	0.0100		乘数是1, 加被乘数
+	0.1101		
	1.0001		右移1位, 得到结果
	0.1000		

加的都是绝对值

小结.

- 乘法运算可用加、移位来实现
- 若 $n=4$, 则加4次、移4次
- 由乘数末位决定被乘数是否与原部分积相加,

然后右移1位形成新部分积, 同时乘数也右移1位(末位移丢), 空出高位存放部分积的低位

- 被乘数只与部分积的高位相加

硬件: 3个寄存器: 存放部分积初值, 存放乘数, 存放被乘数(绝对值部分) 具移位功能
1个全加器: $n+1$ 位长度(否则造成有效数位丢失)

通过两个移位寄存器联动, 加、移位操作交替进行, 就解决了错位相加的问题.

并且始终在左边加, 没有改变运算器长度, 很规整

③ 原码乘法

1) 原码一位乘运算规则

以小数为例,

$$[x]_{\text{原}} = x_0.x_1x_2\dots x_n$$

$$[y]_{\text{原}} = y_0.y_1y_2\dots y_n$$

$$[x \cdot y]_{\text{原}} = (x_0 \oplus y_0). (0.x_1x_2\dots x_n)(0.y_1y_2\dots y_n)$$

用 x^* 来表示 x 的绝对值, 令 $x^* = 0.x_1x_2\dots x_n$

用 y^* 来表示 y 的绝对值, 令 $y^* = 0.y_1y_2\dots y_n$

$$\text{则 } [x \cdot y]_{\text{原}} = (x_0 \oplus y_0). x^* y^*$$

乘积符号位 数值部分:
单独处理 绝对值相乘

2) 原码一位乘递推公式

$$x^* \cdot y^* = x^* (0.y_1y_2\dots y_n)$$

$$= x^* (y_1 \cdot 2^{-1} + y_2 \cdot 2^{-2} + \dots + y_n \cdot 2^{-n}) \quad (n-2 \text{ 个 "})"$$

$$= 2^{-1} (y_1 x^* + 2^{-1} (y_2 x^* + 2^{-1} (y_3 x^* + \dots + 2^{-1} (y_n x^* + 0) \dots))$$

$\underbrace{\hspace{10em}}_{Z_0}$ 共 n 对括号
 $\therefore Z_0$ 加第 1 个位积, $\rightarrow$ 得第 1 次部分积 Z_1
最后一次, Z_{n-1} 加 y 的最高数位的位积, $\rightarrow$ $Z_n = 0$
得最终积 Z_n

$$Z_1 = 2^{-1} (y_1 x^* + Z_0)$$

$$Z_2 = 2^{-1} (y_2 x^* + Z_1)$$

$\vdots$

$$Z_n = 2^{-1} (y_n x^* + Z_{n-1})$$

例. 已知 $x = -0.1110$, $y = 0.1101$, 求 $[x \cdot y]_{\text{原}}$

部分积	乘数	说明
0.0000 + 0.1110	1101	部分积 初态 $Z_0 = 0$ + x^*
0.1110 0.0111 + 0.0000	0110	$\rightarrow 1$ + 0 得 Z_1
0.0111 0.0011 + 0.1110	1011	$\rightarrow 1$ + x^* 得 Z_2
1.0001 0.1000 + 0.1110	1101	$\rightarrow 1$ + x^* 得 Z_3
1.0110 0.1011	110 0110	$\rightarrow 1$ 得 Z_4

结果:

① 乘积符号位: $x_0 \oplus y_0 = 1 \oplus 0 = 1$

② 数值部分按绝对值相乘: $x^* \cdot y^* = 0.10110110$

故 $[x \cdot y]_{\text{原}} = 1.10110110$

特点:

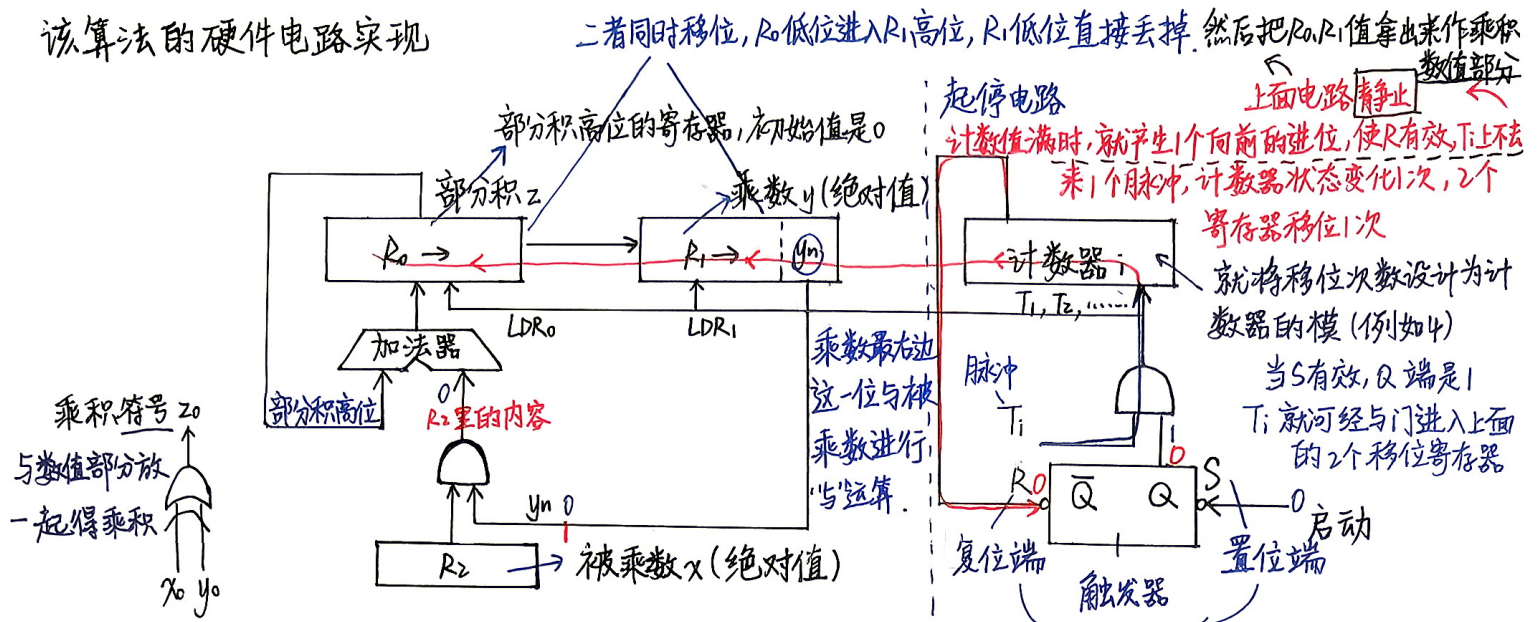
① 绝对值运算

② 通过移位次数判断乘法是否结束:
硬件电路结束工作需要信号,

即停止给移位寄存器脉冲让它停止工作.

③ 逻辑移位.

该算法的硬件电路实现



计数器: 对移位次数进行计数以便判断乘法运算是否结束

当计数器 $i=n$ 时, 计数器 i 的溢出信号使控制触发器 C_x 置0, 关闭时序脉冲 T , 乘法操作结束.

补码乘法

补码一位乘运算规则

以小数为例,

设被乘数 $[x]_{\text{补}} = x_0.x_1x_2 \dots x_n$

设乘数 $[y]_{\text{补}} = y_0.y_1y_2 \dots y_n$

① 被乘数任意, 乘数为正

同原码乘, 但加和移位按补码规则运算

乘积符号自然形成

② 被乘数任意, 乘数为负

乘数 $[y]_{\text{补}}$ 去掉符号位, 操作同①

最后 $+ [-x]_{\text{补}}$ 进行校正

Booth 乘法公式证明

1) 当被乘数 x 符号任意, 乘数 y 符号为正时:

依据补码定义:

以小数为例

$$\because [x]_{\text{补}} = x_0.x_1x_2 \dots x_n = 2 + x = 2^{n+1} + x \pmod{2}$$

正、负都成立

$$[y]_{\text{补}} = 0.y_1y_2 \dots y_n = y$$

$$\therefore [x]_{\text{补}} \cdot [y]_{\text{补}} = \underbrace{2^{n+1}}_{\text{把 } y \text{ 扩大了 } 2^{n+1} \text{ 倍}} \cdot y + x \cdot y = 2 \cdot y_1y_2 \dots y_n + x \cdot y$$

由于 $(y_1y_2 \dots y_n)$ 是 ≥ 1 的正整数, 根据模运算性质 (大于 2 的部分全部丢掉),

有: $2(y_1y_2 \dots y_n) \stackrel{\text{等效于}}{=} 2$

$$\therefore [x]_{\text{补}} \cdot [y]_{\text{补}} = 2 + x \cdot y = [x \cdot y]_{\text{补}} \quad ?$$

正、负都成立

$$\text{因此, } \underbrace{[x \cdot y]_{\text{补}}}_{\text{正}} = [x]_{\text{补}} \cdot [y]_{\text{补}} = [x]_{\text{补}} \cdot \underbrace{y}_{\text{正}}$$

2) 当被乘数 x 符号任意, 乘数 y 符号为负时:

$$[x]_{\text{补}} = x_0.x_1x_2\dots x_n$$

$$[y]_{\text{补}} = 1.y_1y_2\dots y_n = 2 + y \pmod{2}$$

$$\therefore y = [y]_{\text{补}} - 2 = 1.y_1y_2\dots y_n - 2 = 0.y_1y_2\dots y_n - 1$$

$$\therefore x \cdot y = x(0.y_1y_2\dots y_n - 1) = x(0.y_1y_2\dots y_n) - x$$

$$[x \cdot y]_{\text{补}} = [x(0.y_1y_2\dots y_n)]_{\text{补}} - [x]_{\text{补}}$$

$$\text{又} \because 0.y_1y_2\dots y_n > 0$$

$$\therefore [x(0.y_1y_2\dots y_n)]_{\text{补}} = [x]_{\text{补}} \cdot [0.y_1y_2\dots y_n]_{\text{补}} = [x]_{\text{补}} (0.y_1y_2\dots y_n) \quad \text{上一个结论}$$

$$\text{因此, } [x \cdot y]_{\text{补}} = [x]_{\text{补}} (0.y_1y_2\dots y_n) - [x]_{\text{补}}$$

$$= [x]_{\text{补}} (0.y_1y_2\dots y_n - 1)$$

$$= [x]_{\text{补}} \cdot \left(\sum_{i=1}^n y_i 2^{-i} - y_0 \right)$$

$$= [x]_{\text{补}} \cdot y$$

公式展开

为推导出逻辑实现的分步算法, 将上式展开得到各项部分积累加的形式

$$[x \cdot y]_{\text{补}} = [x]_{\text{补}} [y]_{\text{补}}$$

$$= [x]_{\text{补}} (-y_0 + y_1 2^{-1} + y_2 2^{-2} + \dots + y_n 2^{-n}) \quad \text{正、负都成立}$$

$$= [x]_{\text{补}} [-y_0 + (y_1 - y_1 \cdot 2^{-1}) + (y_2 \cdot 2^{-1} - y_2 \cdot 2^{-2}) + \dots + (y_n \cdot 2^{-n+1} - y_n \cdot 2^{-n})]$$

$$= [x]_{\text{补}} [(y_1 - y_0) + (y_2 - y_1) \cdot 2^{-1} + \dots + (y_n - y_{n-1}) \cdot 2^{-n+1} - y_n \cdot 2^{-n}]$$

$$= [x]_{\text{补}} [(y_1 - y_0) + (y_2 - y_1) \cdot 2^{-1} + \dots + (y_n - y_{n-1}) \cdot 2^{-n+1} + (0 - y_n) \cdot 2^{-n}]$$

$$= [x]_{\text{补}} [(y_1 - y_0) + (y_2 - y_1) \cdot 2^{-1} + \dots + (y_n - y_{n-1}) \cdot 2^{-n+1} + \underbrace{(y_{n+1} - y_n) \cdot 2^{-n}}_{\text{增加的附加位, 初值是0}}]$$

$$= [x]_{\text{补}} \cdot \sum_{i=0}^n (y_{i+1} - y_i) \cdot 2^{-i}$$

$$= [x]_{\text{补}} [(y_1 - y_0) + 2^{-1}((y_2 - y_1) + 2^{-1}((y_3 - y_2) + \dots + 2^{-1}(y_{n+1} - y_n) \dots))]$$

$$= (y_1 - y_0) [x]_{\text{补}} + 2^{-1}((y_2 - y_1) [x]_{\text{补}} + 2^{-1}((y_3 - y_2) [x]_{\text{补}} + \dots + 2^{-1}(y_{n+1} - y_n) [x]_{\text{补}} + 0) \dots) \quad \downarrow [Z_0]_{\text{补}}$$

递推公式

$$[Z_0]_{\text{补}} = 0$$

$$[Z_1]_{\text{补}} = 2^{-1}((y_{n+1} - y_n) [x]_{\text{补}} + [Z_0]_{\text{补}}) \quad y_{n+1} = 0$$

$$\vdots$$

$$[Z_n]_{\text{补}} = 2^{-1}((y_2 - y_1) [x]_{\text{补}} + [Z_{n-1}]_{\text{补}})$$

$$[x \cdot y]_{\text{补}} = (y_1 - y_0) [x]_{\text{补}} + [Z_n]_{\text{补}}$$

$[Z_n]_{\text{补}}$

注意! 最后一步不移位!

$[Z_1]_{\text{补}}$

如何实现 $y_{i+1} - y_i$?

y_i	y_{i+1}	$y_{i+1} - y_i$	操作
0	0	0	+0, $\rightarrow 1$
0	1	1	+ $[x]_{\text{补}}$, $\rightarrow 1$
1	0	-1	+ $[-x]_{\text{补}}$, $\rightarrow 1$
1	1	0	+0, $\rightarrow 1$

由此可见:

每次都是在前次部分积的基础上, 由 $(y_{i+1}-y_i)$ 决定对 $[x]_{补}$ 的操作, 再右移1位得新的部分积; 重复进行.

y_{i+1}, y_i 的作用:

开始操作时, 补充一位 y_{n+1} , 使其初始为0

由 $y_i y_{i+1}$ 判断进行何操作, 再由 $y_{i+1} y_i$ 判断下一步进行何操作.

若 $y_i y_{i+1} = 01$, 则 $y_{i+1} - y_i = 1 \rightarrow +[x]_{补}$

若 $y_i y_{i+1} = 10$, 则 $y_{i+1} - y_i = -1 \rightarrow +[-x]_{补}$

若 $y_i y_{i+1} = 11$, 则 $y_{i+1} - y_i = 0 \rightarrow [Z_{n-i}] + 0$, 即: 保持不变

若 $y_i y_{i+1} = 00$, 则 $y_{i+1} - y_i = 0 \rightarrow [Z_{n-i}] + 0$, 即: 保持不变

补码一位乘的运算规则:

① 若 $y_n = y_{n+1}$, 则部分积 $[Z_{n-i}] + 0$, 再右移1位

② 若 $y_n y_{n+1} = 01$, 则部分积 $[Z_{n-i}] + [x]_{补}$, 再右移1位

③ 若 $y_n y_{n+1} = 10$, 则部分积 $[Z_{n-i}] + [-x]_{补}$, 再右移1位

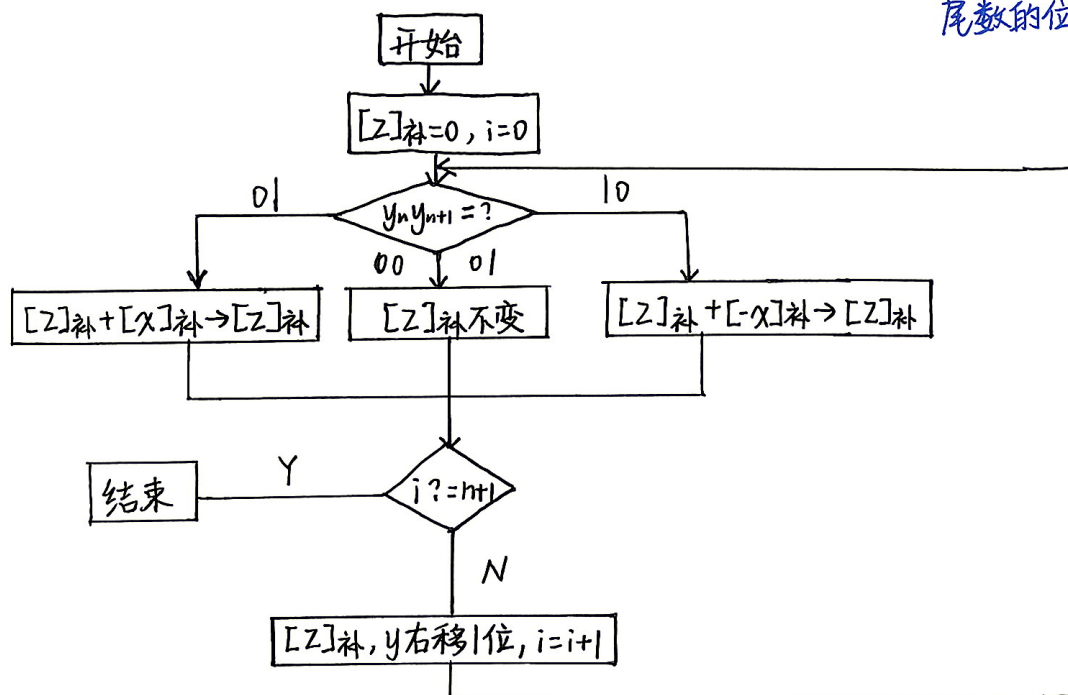
这些判断要重复 $(n+1)$ 步, 但最后1步不! 移! 位!

所得乘积为 $(2n+1)$ 位

符号位

尾数的位数

算法流程图



i 表示移位次数

例. 已知 $x = +0.0011$, $y = -0.1011$, 求 $[x \cdot y]_{补}$

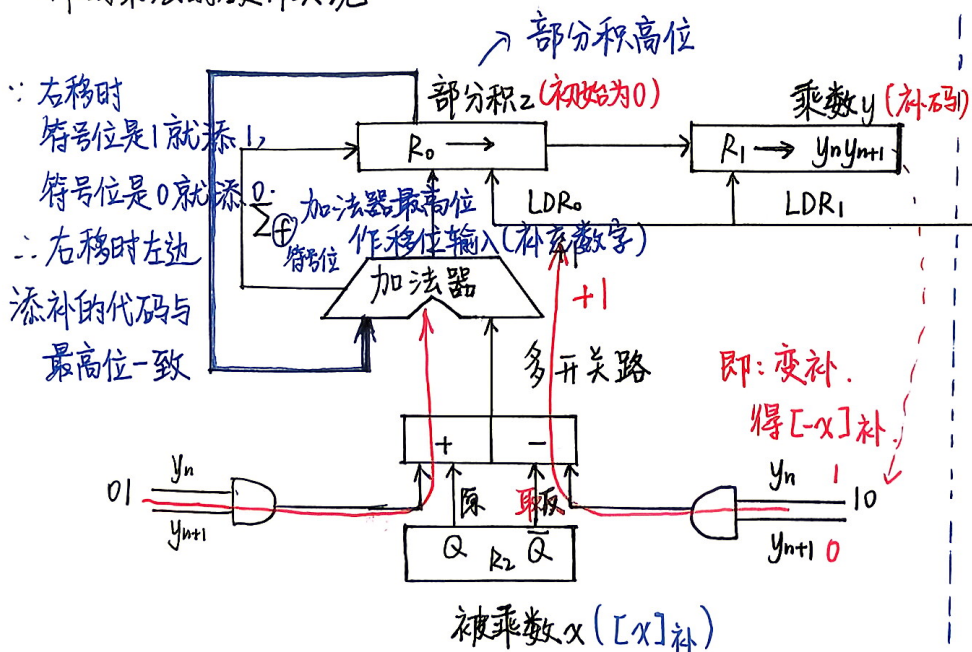
可采用双符号位, 以保证运算过程中有效数位不丢失

解: $[x]_{补} = 0.0011$, $[y]_{补} = 1.0101$, $[-x]_{补} = 1.1101$
变补运算

00.0000 + 11.1101	1.0101	0	$+ [-x]_{补}$
11.1101 11.1110 + 00.0011	1010	1	$\rightarrow 1$ $+ [x]_{补}$
00.0001 00.0000 + 11.1101	1101	0	$\rightarrow 1$ $+ [-x]_{补}$
11.1101 11.1110 + 00.0011	1110	1	$\rightarrow 1$ $+ [x]_{补}$
00.0001 00.0000 + 11.1101	1111	0	$\rightarrow 1$ $+ [-x]_{补}$
11.1101	1111		最后一步不移位.

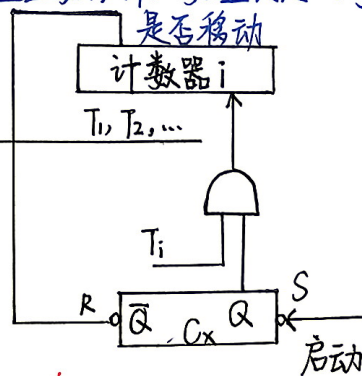
$\therefore [x \cdot y]_{补} = 1.1101111$

补码乘法的硬件实现



起停电路

通过数脉冲的数量决定2个寄存器的内容



移位结束后产生Reset信号,

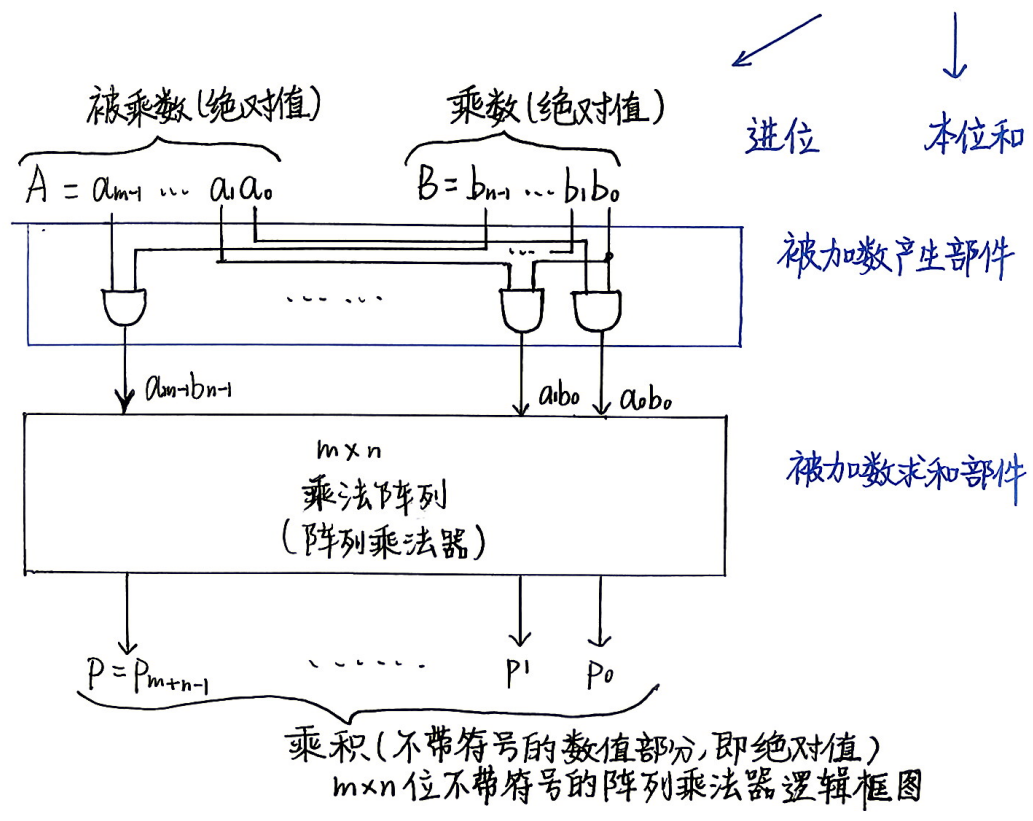
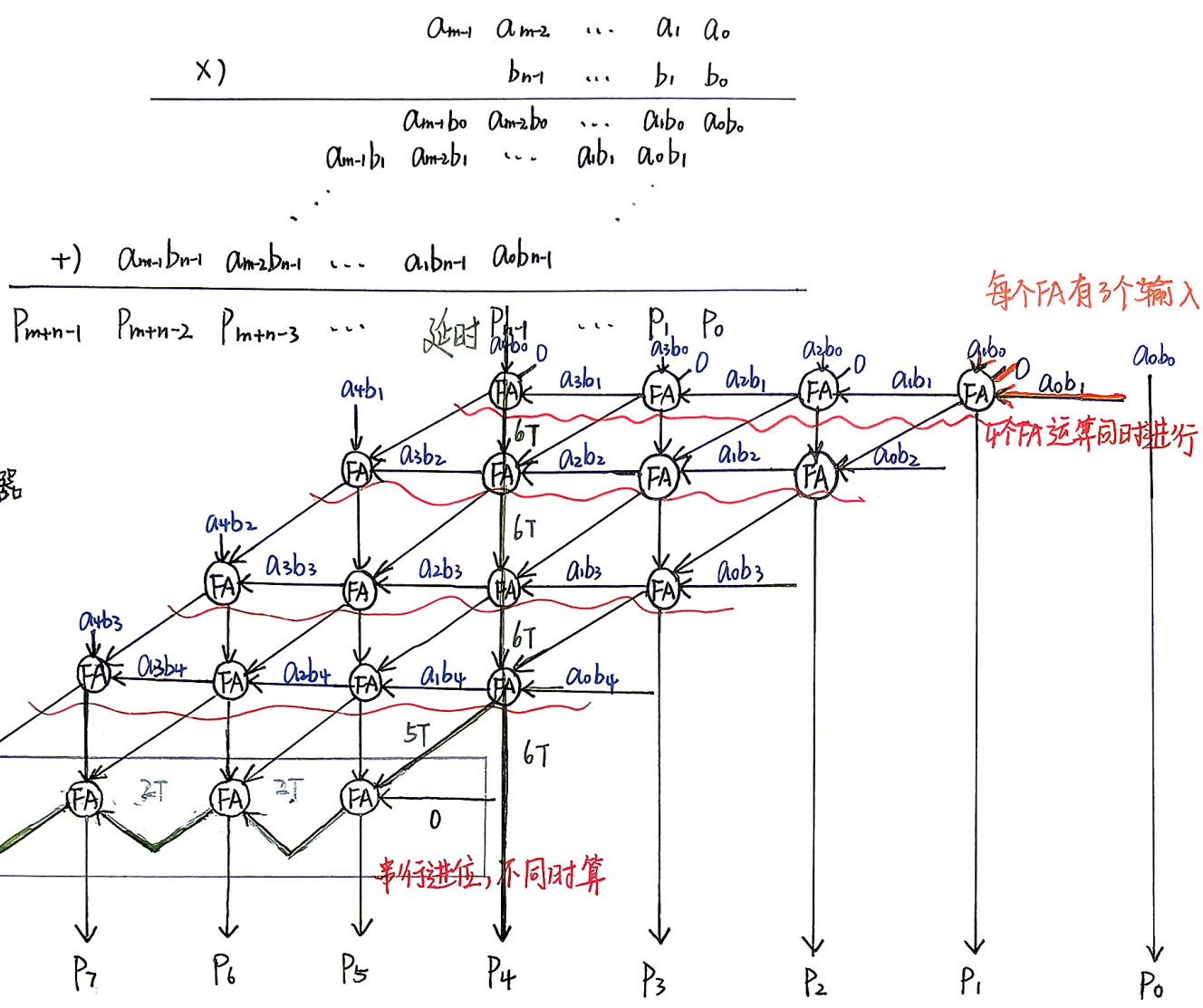
脉冲信号上不去

计数器和2个寄存器不动

最终乘积由2个寄存器内容构成

阵列乘法器 (适用于原码)

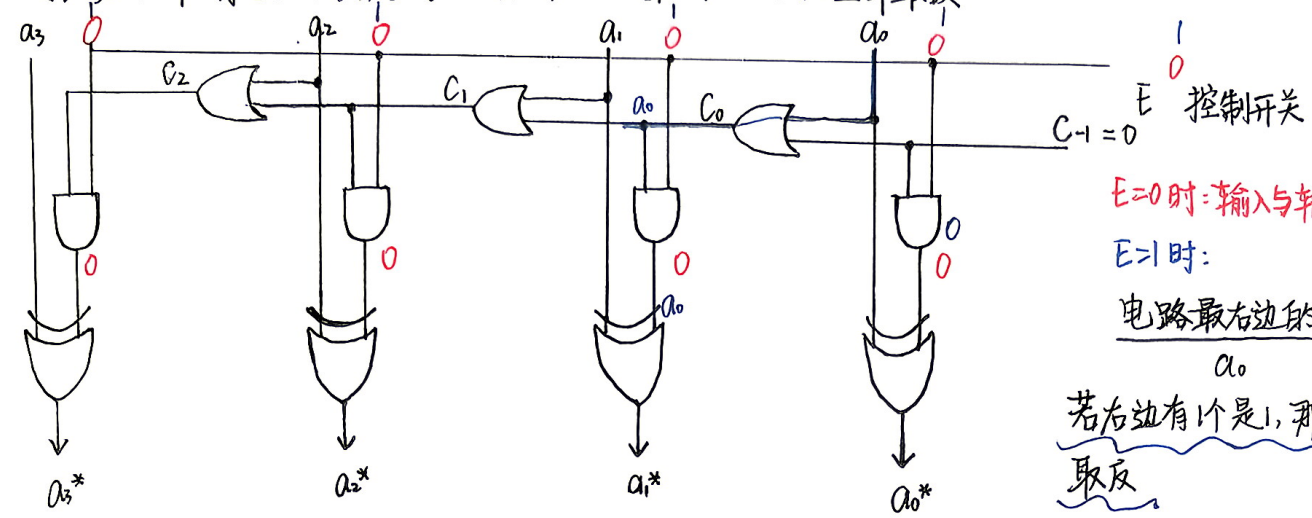
1) 习惯方法运算过程



带符号的阵列乘法器 (适用于补码) 不带符号的更适用于原码.

1) 对2求补器电路

对负数求补, 符号位不变, 数值部分从右向左第一个"1"以后全部取反



$E=0$ 时: 输入与输出不变

$E=1$ 时:
电路最右边的位与输入一致
 a_0

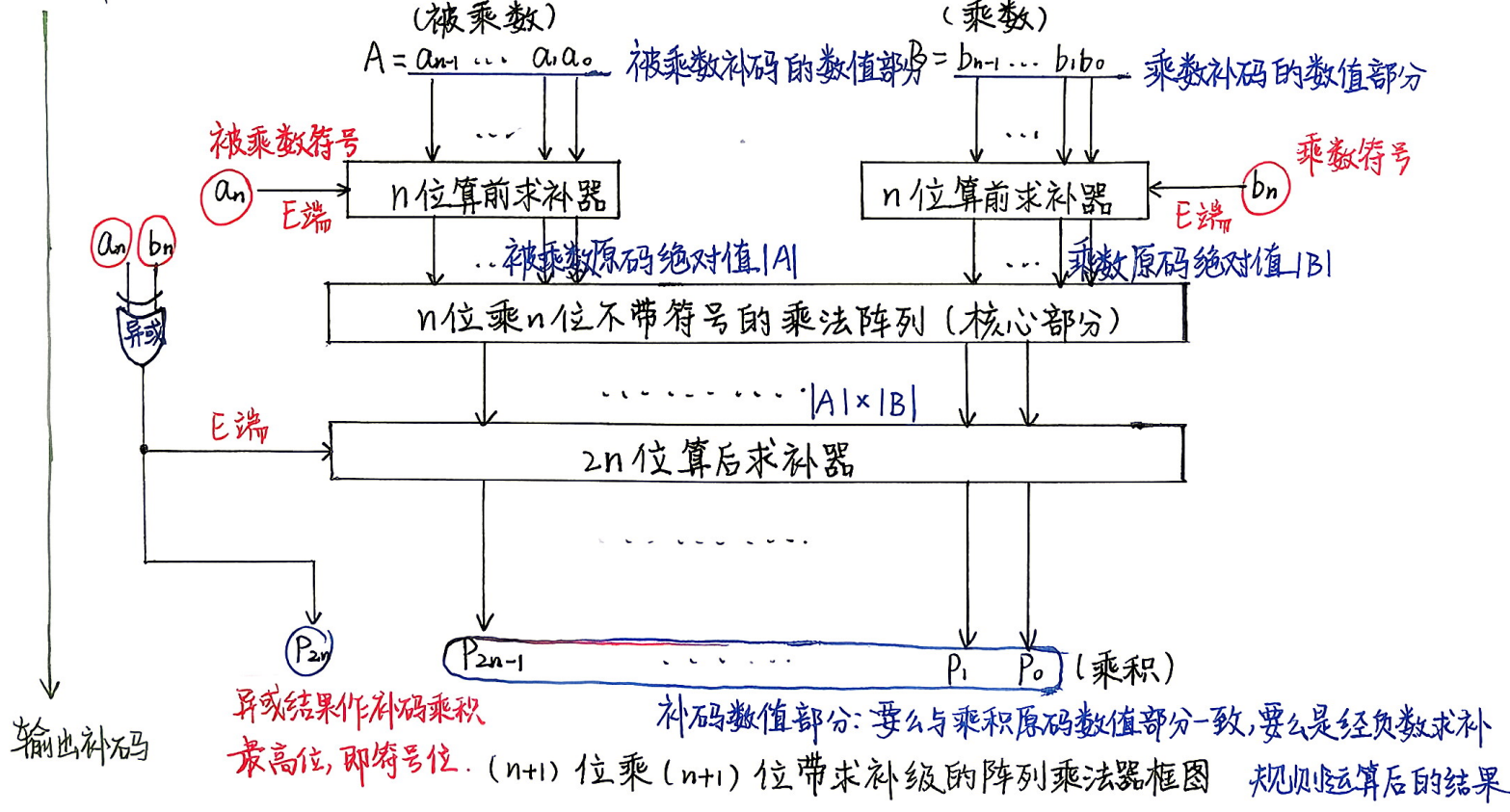
若右边有一个是1, 那么左边全部取反

这就是一个负数的求补规则.

- 如果是补码的话, 工作时将符号位接在E端, 将数值位接在输入端. 这样输出的就是原码的数值部分.
- 如果是原码的话, 输出的就是补码的数值部分.

$$\begin{cases} C_{-1} = 0 \\ C_i = a_i + C_{i-1} \\ a_i^* = a_i \oplus EC_{i-1} \end{cases}$$

输入补码



异或结果作补码乘积最高位, 即符号位.

补码数值部分: 要么与乘积原码数值部分一致, 要么是经负数求补. 规则运算后的结果

可实现带符号数的运算

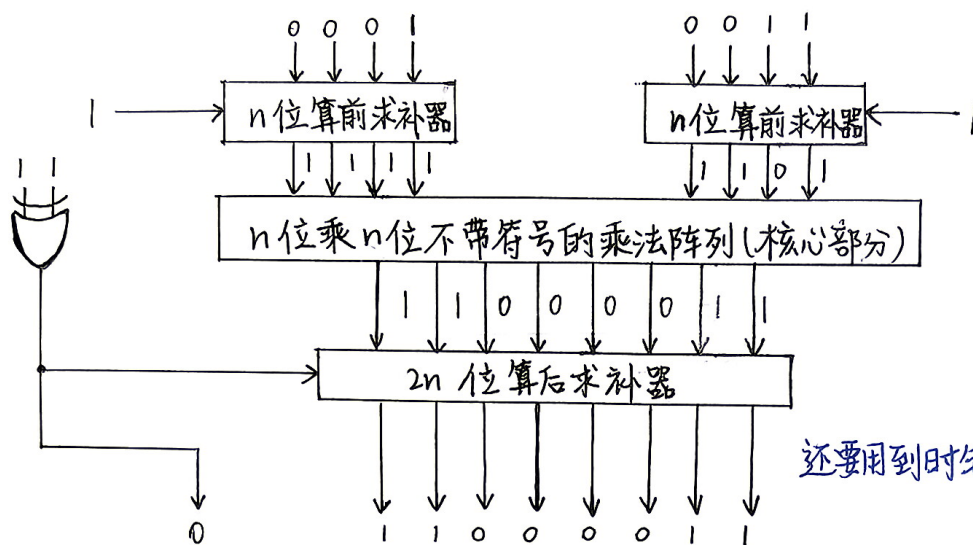
另外, 若两个数都是原码, 工作时两个算前求补器和算后求补器都不工作.

例. 设 $x = -15$, $y = -13$, 用带求补器的补码阵列乘法器求出乘积 $x \cdot y = ?$ 并用十进制数乘法进行验证.

解:

设最高位是符号位, 输入数据用补码表示.

$$[x]_{\text{补}} = 10001, [y]_{\text{补}} = 10011$$



速度快:

速度取决于阵列乘法器
硬件电路的时间延时,
与时序没什么关系

像前面 n 次加法 n 次移位,
还要用到时钟, 有时序, 时间较长, 速度慢

乘积符号位单独运算: $x_f \oplus y_f = 1 \oplus 1 = 0$

尾数部分算前求补器输出是

$$|x| = 1111, |y| = 1101$$

$$\begin{array}{r} x \quad \quad \quad 1111 \\ \quad \quad \quad 1101 \\ \hline \quad \quad \quad 1111 \\ \quad \quad \quad 0000 \\ \quad \quad \quad 1111 \\ + \quad \quad 1111 \\ \hline 11000011 \end{array}$$

乘积符号是 0, 算后求补器输出为 11000011,

最后补码的乘积值为 $[x \cdot y]_{\text{补}} = 011000011$.

补码 = 进制数真值是 $x \cdot y = 0 \times 2^8 + 1 \times 2^7 + 1 \times 2^6 + 1 \times 2^5 + 1 \times 2^4 = (+195)_{\text{补}}$.

$\therefore$ 乘积 $x \cdot y = +195$

验算:

十进制数乘法: $(-15) \times (-13) = +195$

定点除法运算

一、分析笔算除法

$$x = -0.1011 \quad y = 0.1101 \quad \text{求 } x \div y$$

$$\begin{array}{r} 0.1101 \overline{) 0.10110} \\ \underline{0.01101} \\ 0.010010 \\ \underline{0.001101} \\ 0.00010100 \\ \underline{0.00001101} \\ 0.00000111 \end{array}$$

$x \oplus y$ 商符单独处理 $\checkmark$
心算上商 ?

导致需开运算器长, 余数不动低位补0,
度越来越长, 但每次仅后5位参与运算, 而且上商位置不固定?
运算器长度是固定的。

商存放在寄存器中 (移位寄存器)。

$$x \div y = -0.1101, \text{ 余数 } 0.00000111$$

商符由心算求得

二、机器运算与手算不同

1) 在计算机中, 小数点是固定的, 不能简单地采用手算的办法。

为便于机器操作, 除数Y固定不变, 被除数和余数左移 (相当于上面的右移)
(相当于 $\times 2$)

$$\begin{array}{r} 00.1101 \overline{) 00.1001} \\ \underline{01.0010} \\ + 11.0101 \\ \hline 00.0111 \\ \underline{00.1110} \\ + 11.0101 \\ \hline 00.0011 \\ \underline{00.0110} \\ + 11.0101 \\ \hline 00.0001 \end{array}$$

$x = 0.1001, y = 0.1101, [-y]_{\text{补}} = 1.0101$
 $x < y$, 商0
被除数左移1位, 即 $2x$. 而 $2x > y$, 商1
 $-y$, 即 $+ [-y]_{\text{补}}$.
余数 r_1
 r_1 左移1位, 即 $2r_1$. 而 $2r_1 > y$, 商1
 $-y$, 即 $+ [-y]_{\text{补}}$.
余数 r_2
 r_2 左移1位, 即 $2r_2$. 而 $2r_2 < y$, 商0
 r_3 左移1位, 即 $2r_3$. 而 $2r_3 > y$, 商1
 $-y$, 即 $+ [-y]_{\text{补}}$.
余数 r_4

结果与手算相同, 但余数不是真正的余数, 多乘 2^n .

故真正的余数应为 $r_n \times 2^{-n}$, 本题为 0.00000001

2) 机器不会心算, 必须先作减法, 若余数为正, 才知道够减; 若余数为负, 才知道不够减。

不够减时必须恢复原来余数以便继续往下运算的是恢复余数法

3) 要恢复原来的余数, 只要当前余数加除数即可。

但由于恢复余数, 使除法进行过程的步数不固定, 控制复杂。

实际中常用不恢复余数法 (加减交替法)。

运算过程若不够减, 则不必恢复余数, 可根据余数符号继续往下算, 步数固定, 控制简单。

笔算除法与机器除法的比较

笔算除法
商符单独处理
心算上商

余数不动低位补0
减右移1位的除数

2倍字长加法器

上商位置不固定

机器除法
符号位异或形成

$|x| - |y| > 0$ 上商1

$|x| - |y| < 0$ 上商0

余数左移1位低位补0
减除数

1倍字长加法器

在寄存器(可左移)最后一位(最末位)上商

原码除法

以小数为例

$$[x]_{\text{原}} = x_0. x_1 x_2 \dots x_n$$

$$[y]_{\text{原}} = y_0. y_1 y_2 \dots y_n$$

$$[\frac{x}{y}]_{\text{原}} = (\underbrace{x_0 \oplus y_0}_{\substack{\text{商的符号位} \\ \text{单独处理}}}) \cdot \underbrace{\frac{|x|}{|y|}}_{\substack{\text{或写作 } \frac{x^*}{y^*} \\ \text{数值部分为} \\ \text{绝对值相除 } \frac{|x|}{|y|}}}$$

式中 $|x| = 0. x_1 x_2 \dots x_n$ 为 x 的绝对值
 $|y| = 0. y_1 y_2 \dots y_n$ 为 y 的绝对值

为了防止溢出, 有一个约定:

小数定点除法 $|x| < |y|$

(防止商 ≥ 1 , 导致商不是定点小数, 产生溢出)

整数定点除法 $|x| > |y|$

(防止商 < 1 , 导致商不是定点整数, 产生溢出)

$$|x| \neq 0, |y| \neq 0$$

若 $|x| > |y|$, 就给 $|x|$ 乘一个系数

将其变小, 再来除, 最后把商与系数相除?

恢复余数法

被除数减除数, 够减时商1, 不够减时商0

若不够减, 在判断是否商0时已经减掉了除数, 故为保证下次能正确运算, 必须把已减掉的除数加回去恢复余数

例. $x = -0.1011$, $y = -0.1101$, 求 $[\frac{x}{y}]_{\text{原}}$

解: $[x]_{\text{原}} = 1.1011$, $[y]_{\text{原}} = 1.1101$ $[|y|]_{\text{补}} = 0.1101$ $[-|y|]_{\text{补}} = 1.0011$

$x_0 \oplus y_0 = 1 \oplus 1 = 0$
 若被除数比较长, 可将高位放, 低位放
 联动时原本存放在2的数字会逐步进入1参加运算
 两个联动的移位寄存器
 从右边每上完一个商, 它就整体左移, 右边空出来的上新的商

被除数(余数)1	商2	说明
0.1011 + 1.0011	0.0000	+ $[- y]_{\text{补}}$
1.1110 + 0.1101	0	余数为负, 上商0 是结果中小数点左边的这一位恢复余数, + $[y]_{\text{补}}$
0.1011 1.0110 + 1.0011	0	恢复后的余数 左移1位, 即 $\times 2$ + $[- y]_{\text{补}}$
0.1001 1.0010 + 1.0011	01	余数为正, 上商1 左移1位 + $[- y]_{\text{补}}$
0.0101 0.1010 + 1.0011	011	余数为正, 上商1 左移1位 + $[- y]_{\text{补}}$
1.1101 + 0.1101	0110	余数为负, 上商0 恢复余数, + $[y]_{\text{补}}$
0.1010 1.0100 + 1.0011	0110	恢复后的余数 左移1位, 即 $\times 2$ + $[- y]_{\text{补}}$
0.0111	01101	余数为正, 上商1

已精确到小数点后4位

$$\frac{|x|}{|y|} = 0.1101$$

$$\therefore [\frac{x}{y}]_{\text{原}} = 0.1101$$

左移了4位:

$$\text{余数} = 0.0000111$$

不恢复余数法(加减交替法)

由恢复余数法的运算规则:

$$\text{余数 } R_i > 0 \quad \text{上商 } 1 \quad \begin{cases} 2R_i \\ \rightarrow 2R_i - |y| \end{cases} \quad \text{2步}$$

$$\text{余数 } R_i < 0 \quad \text{上商 } 0 \quad \begin{cases} R_i + |y| \\ \rightarrow 2(R_i + |y|) \\ \rightarrow 2(R_i + |y|) - |y| = 2R_i + |y| \end{cases} \quad \begin{matrix} 3\text{步} \\ 2\text{步} \end{matrix}$$

故得不恢复余数法运算规则:

$$\text{上商 } 1 \quad 2R_i - |y|$$

$$\text{上商 } 0 \quad 2R_i + |y|$$

加减交替

例. $x = -0.1011$, $y = -0.1101$, 求 $[\frac{x}{y}]_{\text{原}}$

解:

$$[x]_{\text{原}} = 1.1011, [y]_{\text{原}} = 1.1101, [x]_{\text{补}} = 0.1011, [y]_{\text{补}} = 0.1101, [-y]_{\text{补}} = 1.0011$$

0.1011	0.0000	
+ 1.0011		+ [-y]补
1.1110	0	余数为负, 上商0
1.1100	0	左移1位
+ 0.1101		+ [y]补
0.1001	01	余数为正, 上商1
1.0010	01	左移1位
+ 1.0011		+ [-y]补
0.0101	011	余数为正, 上商1
0.1010	011	左移1位
+ 1.0011		+ [-y]补
1.1101	0110	余数为负, 上商0
1.1010	0110	左移1位
+ 0.1101		+ [y]补
0.0111	01101	余数为正, 上商1

- ① 上商 (n+1) 次
 ② 第1次上商判溢出
 (定点小数第1次相除一定要是0,
 定点整数第1次相除一定要是1)
 ③ 移位 n 次, 加 (n+1) 次
 可以用移位次数判断除法是否结束

$$x_f \oplus y_f = 1 \oplus 1 = 0$$

$$\frac{|x|}{|y|} = 0.1101$$

$$\therefore [\frac{x}{y}]_{\text{原}} = 0.1101$$

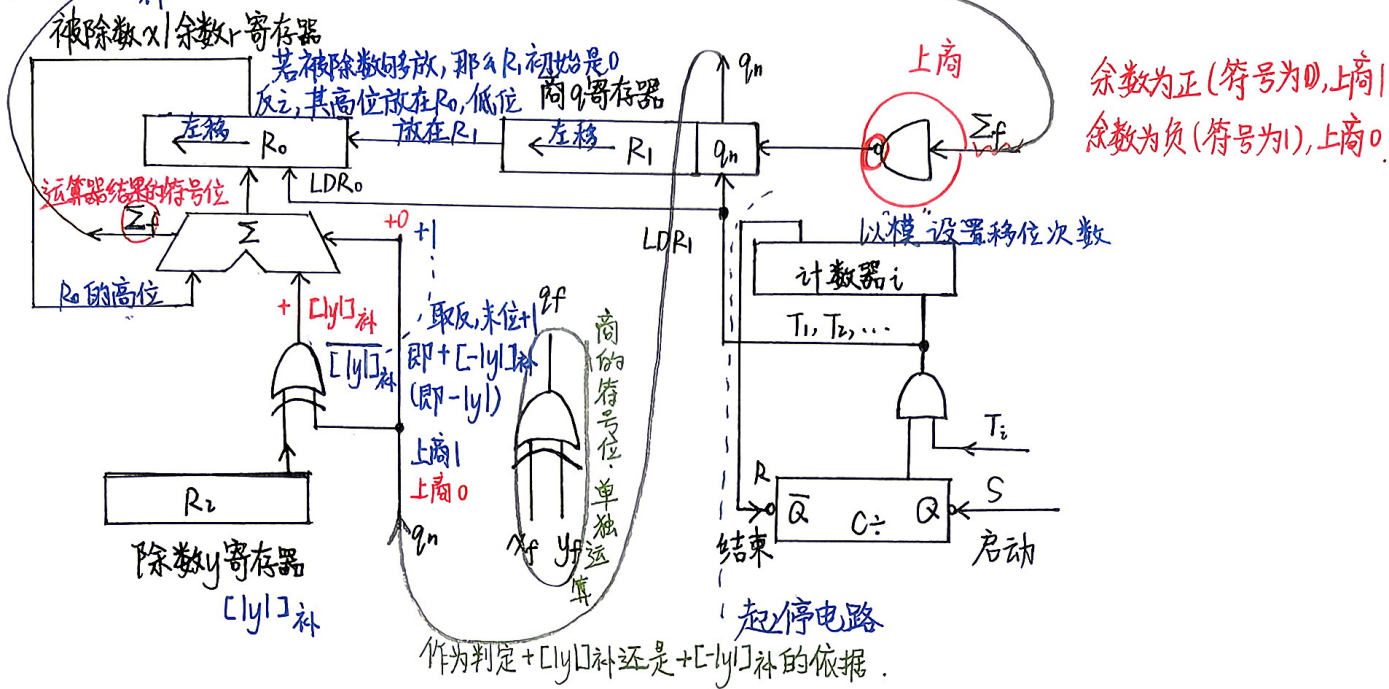
$$\text{余数: } 0.00000111$$

(移34次)

不恢复余数法的硬件实现

引出, 作商寄存器的输入.

放第1次运算前初始时 $|x|$ 补

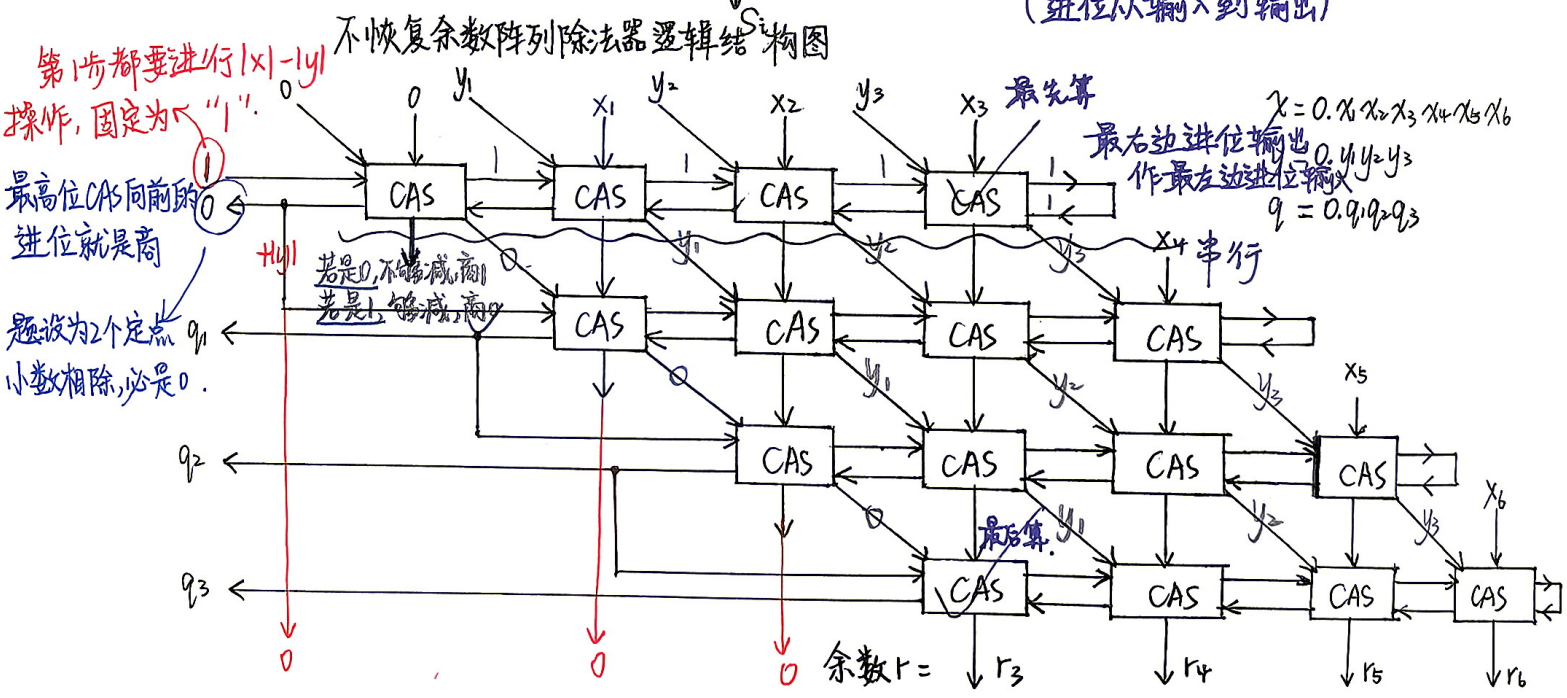
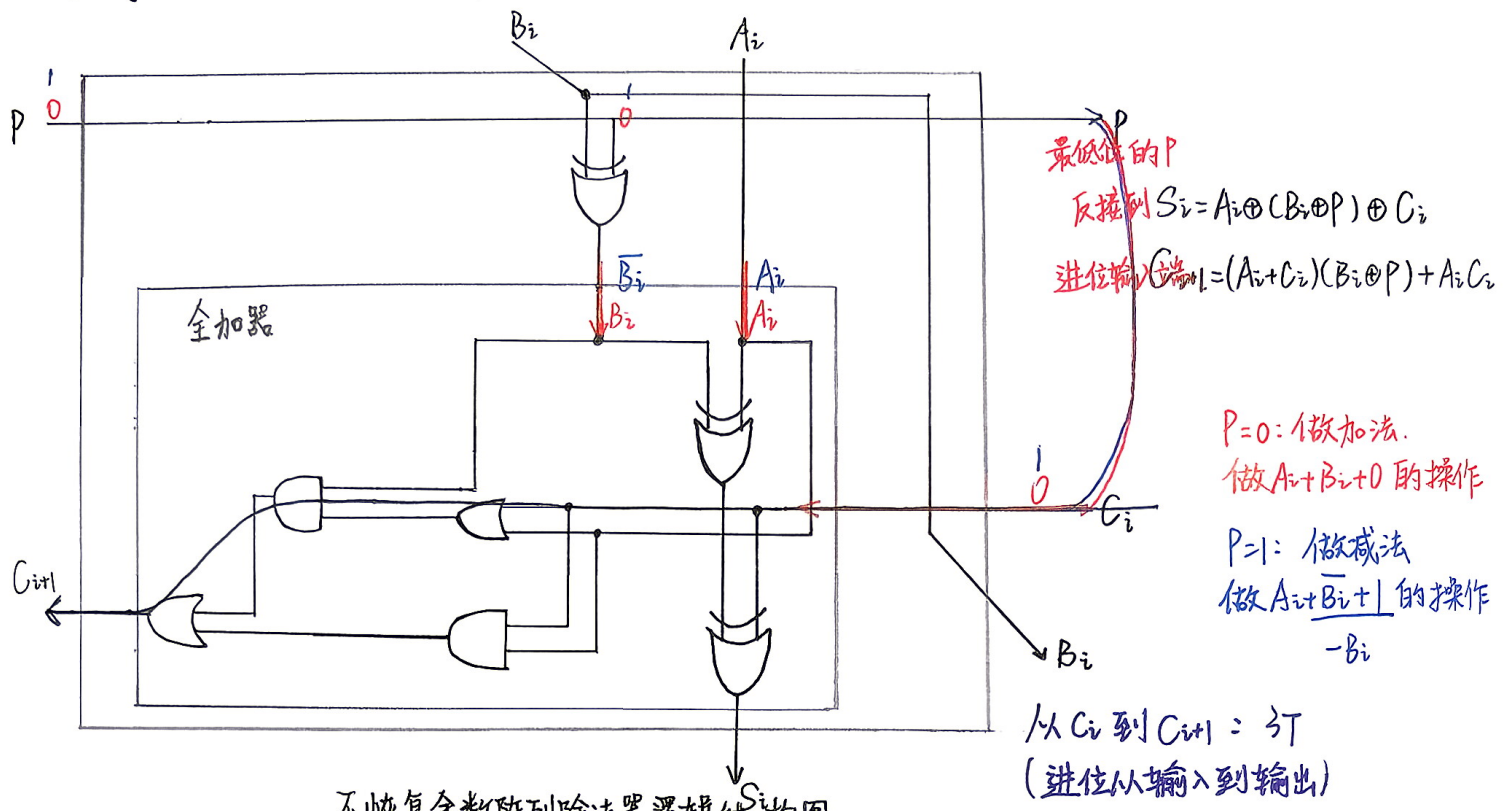


先判断, 再上结果

不恢复余数阵列除法器、补码阵列除法器.....

并行运算部件, 所需的控制线路少, 运算速度快
原码运算的除法器

① 可控加法/减法(CAS)单元



时间延时 = 所有CAS单元的进位信号传递的时间延时

$16 \times 3T$